



Universidad Autónoma del Estado de Hidalgo  
Licenciatura en Ciencias Computacionales  
Automatas y Compiladores  
Docente: Dr.Eduardo Cornejo Velazquez

# REPORTE DE PRÁCTICA FINAL:

## Analizador sintactico

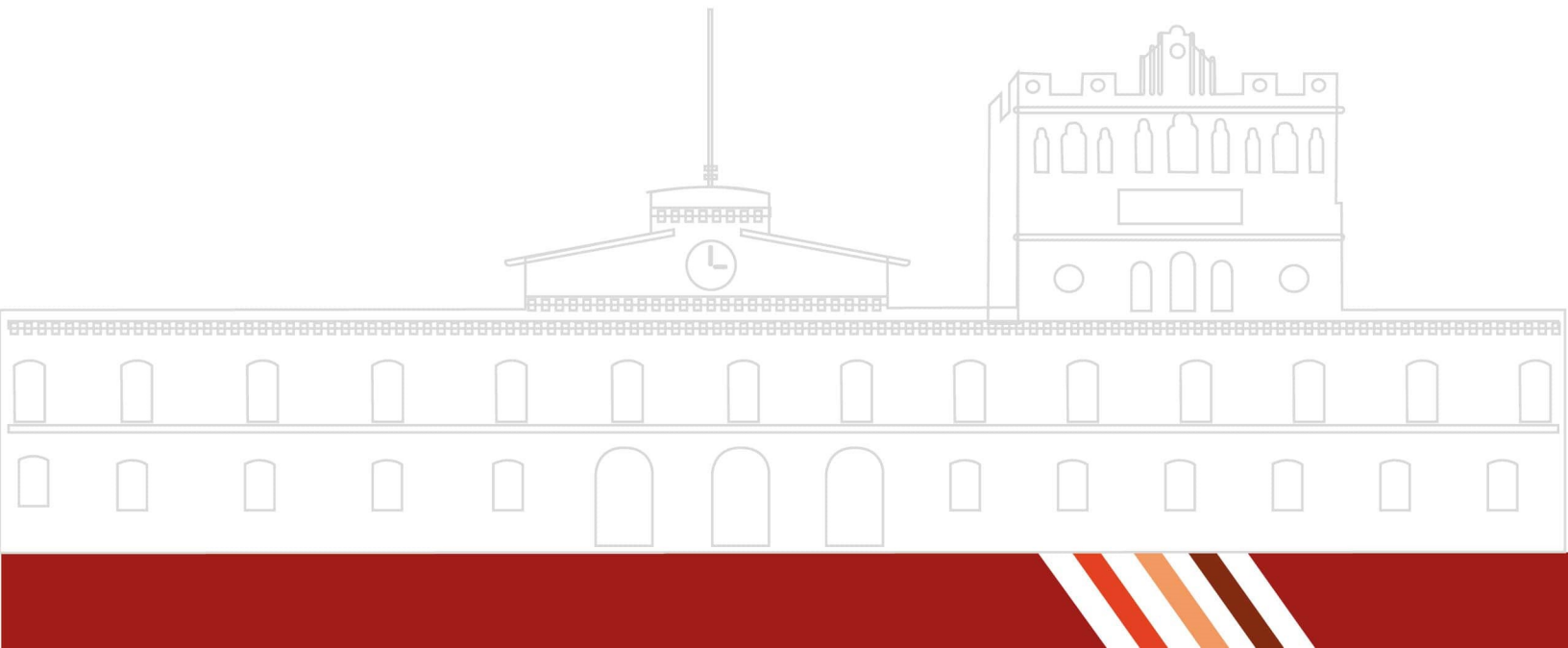
### ALUMNO:

Naomi Romero López

Ángel Ortega Gómez

Adrian Hernández Sánchez

Luis Jose Moreno Ortega



## 1. Introducción

En la presente práctica se desarrolló un analizador sintáctico para instrucciones SQL básicas, específicamente para las sentencias SELECT, INSERT, UPDATE y DELETE, utilizando herramientas de análisis léxico y sintáctico. El propósito principal fue comprender el funcionamiento interno de los compiladores e intérpretes, así como la manera en que un sistema puede reconocer, validar e interpretar la estructura de un lenguaje de consulta.

Para la realización del proyecto se emplearon herramientas como Lex y Yacc dentro del entorno Flex, las cuales permitieron implementar el análisis léxico y sintáctico de las instrucciones SQL. Mediante reglas y gramáticas definidas, el analizador es capaz de identificar tokens, verificar la correcta construcción de las sentencias y detectar errores de sintaxis en las consultas ingresadas.

Esta práctica permitió aplicar conceptos relacionados con lenguajes formales, gramáticas y procesamiento de cadenas, además de reforzar conocimientos sobre el manejo de bases de datos y la importancia de los analizadores en el desarrollo de software. Asimismo, se comprendió cómo herramientas como Flex facilitan la automatización del reconocimiento de estructuras sintácticas en distintos lenguajes de programación y consulta.

## 2. Marco teórico

Los analizadores sintácticos forman parte fundamental de los compiladores e intérpretes, ya que permiten verificar que la estructura de un lenguaje cumpla con las reglas gramaticales establecidas. Su función principal consiste en analizar la secuencia de tokens generada por el analizador léxico y determinar si las instrucciones poseen una sintaxis válida [Aho et al., 2007].

El proceso de compilación generalmente se divide en diferentes etapas, entre las que destacan el análisis léxico y el análisis sintáctico. El análisis léxico se encarga de identificar patrones dentro de una cadena de texto para convertirlos en tokens, mientras que el análisis sintáctico utiliza dichos tokens para construir una representación estructurada de la instrucción ingresada [Louden, 2004].

Para la implementación de analizadores se utilizan herramientas especializadas como Lex y Yacc. Lex es un generador de analizadores léxicos que permite definir expresiones regulares para reconocer patrones dentro de un lenguaje. Por otro lado, Yacc es un generador de analizadores sintácticos basado en gramáticas libres de contexto, utilizado para validar la estructura gramatical de las instrucciones [Levine et al., 1992]. Actualmente, Flex es una versión moderna y optimizada de Lex ampliamente utilizada en sistemas Unix y Linux [Lesk & Schmidt, 1975].

El lenguaje SQL (*Structured Query Language*) es un lenguaje estándar utilizado para la administración y manipulación de bases de datos relacionales. SQL permite realizar operaciones como consultas, inserciones, modificaciones y eliminación de registros mediante sentencias específicas [Elmasri & Navathe, 2016]. Entre las instrucciones más utilizadas se encuentran:

- **SELECT:** utilizada para consultar información almacenada en una base de datos.
- **INSERT:** permite agregar nuevos registros a una tabla.
- **UPDATE:** se emplea para modificar datos existentes.
- **DELETE:** utilizada para eliminar registros de una tabla.

La validación sintáctica de sentencias SQL resulta importante para garantizar que las consultas sean interpretadas correctamente por un sistema gestor de bases de datos. Un analizador sintáctico puede detectar errores en la estructura de las instrucciones antes de que estas sean ejecutadas, evitando inconsistencias y fallos en el procesamiento de información [Silberschatz et al., 2019].

El desarrollo de esta práctica permitió comprender el funcionamiento de los analizadores léxicos y sintácticos mediante el uso de Flex, aplicando gramáticas y reglas para reconocer instrucciones SQL básicas. Asimismo, se reforzaron conocimientos relacionados con compiladores, lenguajes formales y procesamiento de consultas en bases de datos.

## 3. Herramientas empleadas

Para esta practica fueron utilizados los siguientes recursos de software:

1. IDE / Editor: FLEX

## 4. Objetivos

## Objetivo general

Desarrollar un analizador léxico y un analizador sintáctico utilizando Lex y Yacc para reconocer y validar sentencias SQL básicas (**SELECT**, **INSERT**, **UPDATE** y **DELETE**), asegurando la detección de errores y la correcta validación de las gramáticas definidas.

## Objetivos específicos

- Emplear las herramientas conceptuales de lenguajes formales, autómatas y gramáticas libres de contexto para definir la estructura y funcionamiento del analizador SQL.
- Diseñar un programa en Lex que identifique palabras reservadas, identificadores, operadores, símbolos y valores utilizados en las sentencias SQL **SELECT**, **INSERT**, **UPDATE** y **DELETE**.
- Implementar un programa en Yacc que verifique la correcta construcción sintáctica de las sentencias SQL mediante una gramática formal previamente definida.
- Detectar y reportar errores léxicos y sintácticos presentes en las consultas SQL ingresadas por el usuario.
- Comprender el funcionamiento de las herramientas Flex, Lex y Yacc en el desarrollo de analizadores léxicos y sintácticos aplicados a lenguajes de consulta.

## 5. Desarrollo

Para el desarrollo de la práctica se utilizó Flex junto con las herramientas Lex y Yacc para implementar un analizador léxico y sintáctico enfocado en la validación de sentencias SQL básicas. El sistema fue diseñado para reconocer instrucciones **SELECT**, **INSERT**, **UPDATE** y **DELETE**, además de detectar errores sintácticos dentro de las consultas ingresadas.

### 5.1 Implementación del analizador léxico

El analizador léxico fue desarrollado utilizando Lex. Su función principal consiste en leer el archivo de entrada carácter por carácter e identificar los componentes léxicos del lenguaje SQL, transformándolos en tokens que posteriormente son enviados al analizador sintáctico.

Para ello se definieron reglas mediante expresiones regulares capaces de reconocer palabras reservadas, identificadores, números y símbolos especiales. Algunas de las palabras reservadas implementadas fueron:

- **SELECT**
- **FROM**
- **WHERE**
- **INSERT**
- **INTO**
- **VALUES**
- **UPDATE**
- **SET**
- **DELETE**

Asimismo, se definieron tokens para operadores y símbolos especiales como el asterisco (\*), comas, paréntesis y el operador de igualdad (=). También se implementó el reconocimiento de números enteros y de identificadores mediante expresiones regulares.

El siguiente fragmento muestra parte de las reglas implementadas en Lex:

```
"SELECT"      { return SELECT; }
"FROM"        { return FROM; }
"WHERE"       { return WHERE; }
[0-9]+        { return NUMBER; }
[a-zA-Z][a-zA-Z0-9_]* { return ID; }
```

Además, el programa lleva un conteo de líneas mediante la variable `linea`, permitiendo reportar la ubicación aproximada de los errores encontrados durante el análisis.

## 5.2 Implementación del analizador sintáctico

El analizador sintáctico fue desarrollado utilizando Yacc. En esta etapa se definieron las reglas gramaticales necesarias para validar la estructura de las sentencias SQL reconocidas por el analizador léxico.

Se declararon tokens correspondientes a las palabras reservadas y símbolos identificados previamente en Lex:

```
%token SELECT FROM WHERE INSERT INTO VALUES
%token UPDATE SET DELETE
%token ID NUMBER ASTERISK COMMA
%token EQUALS LPAREN RPAREN ERROR
```

Posteriormente se definieron las producciones gramaticales encargadas de validar cada tipo de sentencia SQL. El sistema permite analizar múltiples instrucciones dentro de un mismo archivo de entrada.

La estructura principal del programa se definió de la siguiente manera:

```
programa:
    lista_sentencias
    ;
```

Cada sentencia puede corresponder a una instrucción SELECT, INSERT, UPDATE o DELETE:

```
instruccion:
    select_stmt
    | insert_stmt
    | update_stmt
    | delete_stmt
    ;
```

## 5.3 Validación de sentencias SQL

Para cada tipo de instrucción se definieron reglas específicas. Por ejemplo, la validación de una sentencia SELECT se realizó mediante la siguiente producción:

```
select_stmt:
    SELECT columnas FROM ID opt_where
    ;
```

La instrucción permite seleccionar todas las columnas mediante el símbolo \* o bien especificar una lista de identificadores.

En el caso de las sentencias INSERT, se implementó una gramática que valida la inserción de valores dentro de una tabla:

```
insert_stmt:
    INSERT INTO ID LPAREN lista_ids RPAREN
    VALUES LPAREN lista_vals RPAREN
    ;
```

Asimismo, se implementaron reglas para las instrucciones **UPDATE** y **DELETE**, incluyendo una cláusula opcional **WHERE** para establecer condiciones.

## 5.4 Manejo de errores

El sistema cuenta con detección de errores sintácticos mediante la producción **error** proporcionada por Yacc. Cuando una sentencia no cumple con la gramática definida, el programa genera un mensaje indicando la línea aproximada donde ocurrió el error.

```
| error {
    fprintf(yyout,
        "[Línea %d]: Error de sintaxis.\n", linea);
    yyerrok;
    yyclearin;
}
```

Si la instrucción es válida, el sistema muestra un mensaje de confirmación en el archivo de salida:

```
[Línea X]: Sentencia válida.
```

## 5.5 Ejecución del programa

El programa fue configurado para leer las sentencias SQL desde un archivo llamado **entrada.txt** y almacenar los resultados en un archivo **salida.txt**. Finalmente, se utilizó la función **yyparse()** para iniciar el proceso de análisis sintáctico.

Con la implementación de este sistema fue posible integrar el análisis léxico y sintáctico en una sola aplicación capaz de reconocer y validar instrucciones SQL básicas mediante el uso de Flex, Lex y Yacc.

## 5.6 Compilación del proyecto

Una vez implementados el analizador léxico y el analizador sintáctico, se procedió a realizar la compilación del proyecto dentro del entorno Flex utilizando las herramientas Lex y Yacc.

El proceso de compilación se realizó en tres etapas principales:

1. Primero se compiló el archivo correspondiente al analizador léxico desarrollado en Lex. En esta etapa se generó automáticamente el código fuente encargado de reconocer los tokens definidos mediante expresiones regulares.
2. Posteriormente se compiló el archivo del analizador sintáctico desarrollado en Yacc. Durante este proceso se generaron las tablas y reglas necesarias para validar la gramática definida para las sentencias SQL.
3. Finalmente, se utilizó la opción de construcción conjunta **Lex + Yacc**, la cual permitió integrar ambos analizadores en un solo programa ejecutable. De esta manera, el analizador léxico y el sintáctico trabajaron de forma conjunta para procesar las instrucciones SQL ingresadas.

Después de la compilación, el programa ejecutó el análisis de las consultas almacenadas en el archivo `entrada.txt`, generando los resultados y mensajes correspondientes en el archivo `salida.txt`. Con ello se comprobó el correcto funcionamiento del sistema y la integración adecuada entre Lex y Yacc.

## 5.6 Código del analizador léxico

El analizador léxico fue implementado utilizando Lex. En esta sección se definieron las reglas encargadas de reconocer las palabras reservadas, operadores, identificadores y símbolos utilizados en las sentencias SQL.

### 5.7.1 Declaraciones iniciales

En esta parte se incluyen las librerías necesarias y se declara la variable utilizada para llevar el control de líneas analizadas.

```
%{
#include "y.tab.h"
#include <stdio.h>
int linea = 1;
%}
```

```
%option noyywrap
```

### 5.7.2 Definición de tokens

Se definieron las expresiones regulares encargadas de reconocer las palabras reservadas de SQL, operadores relacionales, números, cadenas de texto e identificadores.

```
%%
"SELECT"    { return SELECT; }
"FROM"      { return FROM; }
"WHERE"     { return WHERE; }
"INSERT"    { return INSERT; }
"INTO"      { return INTO; }
"VALUES"    { return VALUES; }
"UPDATE"    { return UPDATE; }
"SET"       { return SET; }
"DELETE"    { return DELETE; }

"*"         { return ASTERISK; }
", "        { return COMMA; }
"="         { return EQUALS; }
"("         { return LPAREN; }
")"         { return RPAREN; }
";"         { return SEMICOLON; }

">"         { return GREATER; }
"<"         { return LESS; }

[0-9]+      { return NUMBER; }
```

```

[a-zA-Z][a-zA-Z0-9]* { return ID; }

\'[^\']*\' { return STRING; }

"\n"      { linea++; }

[ \t\r]+  ; /* Ignorar espacios */

.         { return ERROR; }
%%

```

El analizador reconoce distintos tipos de datos y operadores utilizados en las consultas SQL. También se implementó el reconocimiento de cadenas de texto delimitadas por comillas simples y operadores relacionales como mayor que y menor que.

## 5.7 Código del analizador sintáctico

El analizador sintáctico fue desarrollado utilizando Yacc. En esta etapa se definieron las gramáticas encargadas de validar la estructura de las sentencias SQL.

### 5.8.1 Declaraciones iniciales

Se incluyeron las librerías necesarias, variables globales y funciones utilizadas para el manejo de errores y lectura de archivos.

```

%{
#include <stdio.h>
#include <stdlib.h>

extern int linea;
extern FILE *yyin;
extern FILE *yyout;
extern char *yytext;

int huboErrores = 0;

void yyerror(const char *s);
int yylex();
void yyrestart(FILE *input_file);
%}

```

### 5.8.2 Declaración de tokens

Se declararon todos los tokens que serán utilizados por las reglas gramaticales.

```

%token SELECT FROM WHERE INSERT INTO VALUES
%token UPDATE SET DELETE

```

```
%token ID NUMBER ASTERISK COMMA
%token EQUALS LPAREN RPAREN ERROR
%token SEMICOLON STRING GREATER LESS
```

### 5.8.3 Gramática principal

La gramática principal permite analizar múltiples sentencias SQL dentro de un mismo archivo.

```
%%
programa:
    lista_sentencias
    ;

lista_sentencias:
    sentencia
    | lista_sentencias sentencia
    ;
```

### 5.8.4 Validación de sentencias

Cada sentencia puede ser válida o contener errores sintácticos. El sistema genera mensajes dependiendo del resultado del análisis.

```
sentencia:
    instruccion SEMICOLON
    {
        fprintf(yyout,
            "[Línea %d]: Sentencia válida.\n",
            linea);
    }

    | error SEMICOLON
    {
        fprintf(yyout,
            "[Línea %d]: Sentencia inválida.\n",
            linea);

        huboErrores = 1;
        yyerrok;
    }
    ;
```

### 5.8.5 Reglas gramaticales de SQL

Se definieron reglas específicas para validar las instrucciones **SELECT**, **INSERT**, **UPDATE** y **DELETE**.

```
instruccion:
    select_stmt
```

```

    | insert_stmt
    | update_stmt
    | delete_stmt
    ;

select_stmt:
    SELECT columnas FROM ID opt_where
    ;

insert_stmt:
    INSERT INTO ID LPAREN lista_ids RPAREN
    VALUES LPAREN lista_vals RPAREN
    ;

update_stmt:
    UPDATE ID SET asignaciones opt_where
    ;

delete_stmt:
    DELETE FROM ID opt_where
    ;

```

### 5.8.6 Cláusulas auxiliares

También se implementaron reglas auxiliares para listas de identificadores, valores y condiciones opcionales.

```

columnas:
    ASTERISK
    | lista_ids
    ;

lista_ids:
    ID
    | lista_ids COMMA ID
    ;

lista_vals:
    valor
    | valor COMMA lista_vals
    ;

valor:
    ID
    | NUMBER
    | STRING
    ;

operador:
    EQUALS
    | GREATER
    | LESS
    ;

```

```

opt_where:
    WHERE ID operador valor
    |
    ;

asignaciones:
    ID EQUALS valor
    | asignaciones COMMA ID EQUALS valor
    ;
%%

```

### 5.8.7 Manejo de errores

El sistema implementa una función personalizada para mostrar mensajes de error sintáctico indicando la línea y el token donde ocurrió el problema.

```

void yyerror(const char *s) {
    huboErrores = 1;

    fprintf(yyout,
        "[Línea %d] Error sintáctico cerca de '%s': %s\n",
        linea,
        yytext,
        s
    );
}

```

### 5.8.8 Función principal

La función principal se encargó de abrir los archivos de entrada, ejecutar el análisis y generar el archivo de resultados.

```

int main(int argc, char *argv[]) {

    FILE *archivo1, *archivo2;
    linea = 1;

    yyout = fopen("resultado.txt", "w");

    archivo1 = fopen("sqlcorrecto.txt", "r");

    yyin = archivo1;
    yyparse();

    archivo2 = fopen("sqlincorrecto.txt", "r");

    yyin = archivo2;
    yyparse();
}

```

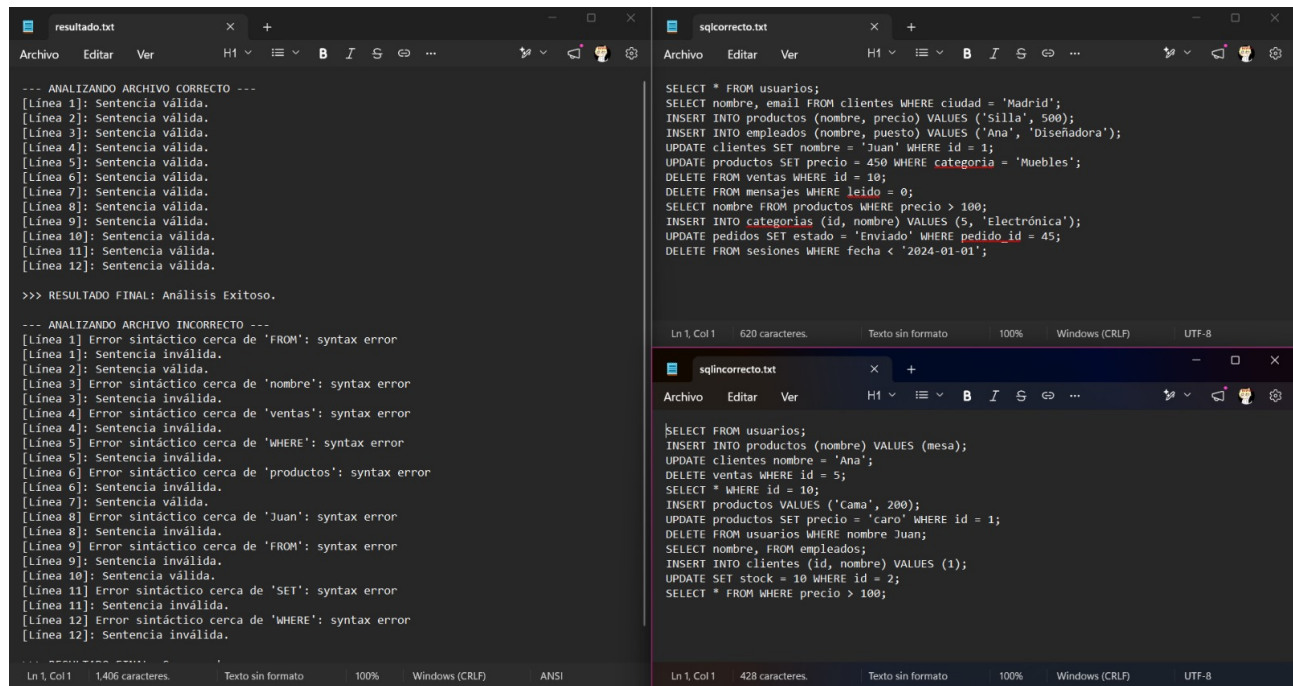
```
    fclose(yyout);  
  
    return 0;  
}
```

Con estas implementaciones se logró integrar correctamente el análisis léxico y sintáctico de sentencias SQL utilizando Flex, Lex y Yacc.

## 6. Resultados

En esta sección se presentan los resultados obtenidos tras la ejecución del analizador sintáctico desarrollado con Lex y Yacc. Se procesaron dos conjuntos de pruebas: uno compuesto por expresiones correctas y otro por expresiones con errores sintácticos, con el objetivo de validar el funcionamiento del sistema.

En la Figura 1 se observan los archivos ingresados así como los resultados obtenidos correspondientemente. En la Figura 2 se muestra el archivo de entrada que contiene las sentencias SQL correctas utilizadas para



```
--- ANALIZANDO ARCHIVO CORRECTO ---
[Línea 1]: Sentencia válida.
[Línea 2]: Sentencia válida.
[Línea 3]: Sentencia válida.
[Línea 4]: Sentencia válida.
[Línea 5]: Sentencia válida.
[Línea 6]: Sentencia válida.
[Línea 7]: Sentencia válida.
[Línea 8]: Sentencia válida.
[Línea 9]: Sentencia válida.
[Línea 10]: Sentencia válida.
[Línea 11]: Sentencia válida.
[Línea 12]: Sentencia válida.

>>> RESULTADO FINAL: Análisis Exitoso.

--- ANALIZANDO ARCHIVO INCORRECTO ---
[Línea 1] Error sintáctico cerca de 'FROM': syntax error
[Línea 1]: Sentencia inválida.
[Línea 2]: Sentencia válida.
[Línea 3] Error sintáctico cerca de 'nombre': syntax error
[Línea 3]: Sentencia inválida.
[Línea 4] Error sintáctico cerca de 'ventas': syntax error
[Línea 4]: Sentencia inválida.
[Línea 5] Error sintáctico cerca de 'WHERE': syntax error
[Línea 5]: Sentencia inválida.
[Línea 6] Error sintáctico cerca de 'productos': syntax error
[Línea 6]: Sentencia inválida.
[Línea 7]: Sentencia válida.
[Línea 8] Error sintáctico cerca de 'Juan': syntax error
[Línea 8]: Sentencia inválida.
[Línea 9] Error sintáctico cerca de 'FROM': syntax error
[Línea 9]: Sentencia inválida.
[Línea 10]: Sentencia válida.
[Línea 11] Error sintáctico cerca de 'SET': syntax error
[Línea 11]: Sentencia inválida.
[Línea 12] Error sintáctico cerca de 'WHERE': syntax error
[Línea 12]: Sentencia inválida.
```

```
SELECT * FROM usuarios;
SELECT nombre, email FROM clientes WHERE ciudad = 'Madrid';
INSERT INTO productos (nombre, precio) VALUES ('Silla', 500);
INSERT INTO empleados (nombre, puesto) VALUES ('Ana', 'Diseñadora');
UPDATE clientes SET nombre = 'Juan' WHERE id = 1;
UPDATE productos SET precio = 450 WHERE categoria = 'Muebles';
DELETE FROM ventas WHERE id = 10;
DELETE FROM mensajes WHERE leído = 0;
SELECT nombre FROM productos WHERE precio > 100;
INSERT INTO categorias (id, nombre) VALUES (5, 'Electrónica');
UPDATE pedidos SET estado = 'Enviado' WHERE pedido_id = 45;
DELETE FROM sesiones WHERE fecha < '2024-01-01';
```

```
SELECT FROM usuarios;
INSERT INTO productos (nombre) VALUES (mesa);
UPDATE clientes nombre = 'Ana';
DELETE ventas WHERE id = 5;
SELECT * WHERE id = 10;
INSERT productos VALUES ('Cama', 200);
UPDATE productos SET precio = 'caro' WHERE id = 1;
DELETE FROM usuarios WHERE nombre Juan;
SELECT nombre, FROM empleados;
INSERT INTO clientes (id, nombre) VALUES (1);
UPDATE SET stock = 10 WHERE id = 2;
SELECT * FROM WHERE precio > 100;
```

Figure 1: Resultados

comprobar el funcionamiento adecuado del analizador sintáctico, así como los resultados generados después del análisis.

En la Figura 3 se presenta el archivo de entrada con sentencias SQL incorrectas utilizadas para verificar la detección de errores sintácticos por parte del analizador, junto con los resultados obtenidos durante la ejecución.

```
SELECT * FROM usuarios;
SELECT nombre, email FROM clientes WHERE ciudad = 'Madrid';
INSERT INTO productos (nombre, precio) VALUES ('Silla', 500);
INSERT INTO empleados (nombre, puesto) VALUES ('Ana', 'Diseñadora');
UPDATE clientes SET nombre = 'Juan' WHERE id = 1;
UPDATE productos SET precio = 450 WHERE categoria = 'Muebles';
DELETE FROM ventas WHERE id = 10;
DELETE FROM mensajes WHERE leído = 0;
SELECT nombre FROM productos WHERE precio > 100;
INSERT INTO categorias (id, nombre) VALUES (5, 'Electrónica');
UPDATE pedidos SET estado = 'Enviado' WHERE pedido_id = 45;
DELETE FROM sesiones WHERE fecha < '2024-01-01';|
```

Figure 2: Archivo de pruebas con sentencias SQL correctas

```
SELECT FROM usuarios;
INSERT INTO productos (nombre) VALUES (mesa);
UPDATE clientes nombre = 'Ana';
DELETE ventas WHERE id = 5;
SELECT * WHERE id = 10;
INSERT productos VALUES ('Cama', 200);
UPDATE productos SET precio = 'caro' WHERE id = 1;
DELETE FROM usuarios WHERE nombre Juan;
SELECT nombre, FROM empleados;
INSERT INTO clientes (id, nombre) VALUES (1);
UPDATE SET stock = 10 WHERE id = 2;
SELECT * FROM WHERE precio > 100;
```

Figure 3: Archivo de pruebas con sentencias SQL incorrectas

## Conclusiones

En esta práctica se logró desarrollar un analizador léxico y sintáctico para sentencias SQL básicas utilizando las herramientas Flex, Lex y Yacc. El sistema implementado fue capaz de reconocer y validar instrucciones **SELECT**, **INSERT**, **UPDATE** y **DELETE**, además de detectar errores sintácticos presentes en las consultas ingresadas.

Durante el desarrollo del proyecto se aplicaron conceptos relacionados con compiladores, autómatas y gramáticas libres de contexto, permitiendo comprender el funcionamiento interno de los analizadores léxicos y sintácticos. Asimismo, se reforzó el uso de expresiones regulares para el reconocimiento de tokens y la definición de reglas gramaticales para validar estructuras formales.

La práctica también permitió entender la importancia de herramientas como Lex y Yacc en la automatización del análisis de lenguajes, facilitando la construcción de programas capaces de interpretar instrucciones de manera eficiente. Además, se comprobó la integración entre ambos analizadores mediante el proceso de compilación y ejecución dentro del entorno Flex.

Finalmente, el proyecto cumplió con los objetivos establecidos, ya que el analizador desarrollado funcionó correctamente al procesar sentencias SQL válidas e inválidas, generando mensajes adecuados para cada caso y demostrando la utilidad de los analizadores en aplicaciones relacionadas con bases de datos y procesamiento de lenguajes.

## References

- [Aho et al., 2007] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson.
- [Elmasri & Navathe, 2016] Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
- [Lesk & Schmidt, 1975] Lesk, M. E., & Schmidt, E. (1975). *Lex - A Lexical Analyzer Generator*. Bell Laboratories.
- [Levine et al., 1992] Levine, J. R., Mason, T., & Brown, D. (1992). *Lex & Yacc*. O'Reilly Media.
- [Louden, 2004] Loudon, K. C. (2004). *Compiler Construction: Principles and Practice*. Cengage Learning.
- [Silberschatz et al., 2019] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill.